

real-time-systems-course Guía 3 de Trabajos Prácticos

practical exercises and projects of the real-time systems course.

Desafío 1

Funcionamiento: Tener servicios funcionando para la recepción RxTask y la transferencia TxTask de datos por puerto serie via Puerto COM Virtual (Virtual COM Port).

```
void vTareaBoton(void *pvParameters){
    const char *pcMSG = "[EVENTO] Pulsador accionado\r\n";

    while(1){

        if(xSemaphoreTake(xButton_Sem, portMAX_DELAY) == pdPASS){

            HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_14);
            usb_transmit_buffer_safe(pcMSG);

            // Anti Bouncing
            xSemaphoreTake(xButton_Sem, 0);
            vTaskDelay(pdMS_TO_TICKS(100));

        }
    }
}

uint8_t usb_transmit_buffer_safe(const char *pcString){
    uint8_t status;
    uint16_t len = 0U;

    if(xSemaphoreTake(xUSB_Tx_Mutex, portMAX_DELAY) == pdTRUE){

        while(pcString[len] != '\0') len++;
        do {
            status = CDC_Transmit_FS((uint8_t*)pcString, len);

            if(status == USB_D_BUSY) {
                // Liberamos el CPU para despues volver a iterar

                vTaskDelay(pdMS_TO_TICKS(1));
            }
        } while(status == USB_D_BUSY);

        // Liberamos el Mutex
        xSemaphoreGive(xUSB_Tx_Mutex);
    }
    return USB_D_OK;
}
```

Desafío 2

Preguntas: Es correcto el funcionamiento observado? Los caracteres se muestran en el orden que se enviaron? Los mensajes por terminal están corruptos? Porque?

Análisis:

```
uint8_t usb_transmit_buffer_safe(const char *pcString){
    uint8_t status;
    uint16_t len = 0U;

    if(xSemaphoreTake(xUSB_Tx_Mutex, portMAX_DELAY) == pdTRUE){

        while(pcString[len] != '\0') len++; // Standard de C
        do {
            status = CDC_Transmit_FS((uint8_t*)pcString, len);

            if(status == USBD_BUSY) {
                // Liberamos el CPU para despues volver a iterar

                vTaskDelay(pdMS_TO_TICKS(1));
            }
        } while(status == USBD_BUSY);

        // Liberamos el Mutex
        xSemaphoreGive(xUSB_Tx_Mutex);
    }
    return USBD_OK;
}

void vTareaProcesadora(void *pvParameters) {
    char rxChar;
    char txBuffer[80]; // Buffer local para armar el mensaje
    UBaseType_t pendingCount; // Variable para almacenar la cantidad 'N'

    while(1) {

        if(xQueueReceive(xUSB_Rx_Queue, &rxChar, portMAX_DELAY) == pdPASS) {

            HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_SET);
            vTaskDelay(pdMS_TO_TICKS(50));
            HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_RESET);

            pendingCount = uxQueueMessagesWaiting(xUSB_Rx_Queue);

            snprintf(txBuffer, sizeof(txBuffer),
                "Recibido: '%c' - Caracteres pendientes: %u \r\n",
                rxChar, (unsigned int)pendingCount);

            usb_transmit_buffer_safe(txBuffer);
        }
    }
}
```

Desafío 3

Preguntas: Observar la el orden en el que aparecen los distintos mensajes de A y B. Son equitativos? Porque? Repensar que mecanismo puede implementarse para intentar que siempre se de la secuencia A - B - A - B....

Análisis:

```
uint8_t usb_transmit_buffer_safe(const char *pcString){
    uint8_t status;
    uint16_t len = 0U;

    if(xSemaphoreTake(xUSB_Tx_Mutex, portMAX_DELAY) == pdTRUE){

        while(pcString[len] != '\0') len++; // Standard de C
        do {
            status = CDC_Transmit_FS((uint8_t*)pcString, len);

            if(status == USB_D_BUSY) {
                // Liberamos el CPU para despues volver a iterar

                vTaskDelay(pdMS_TO_TICKS(1));
            }
        } while(status == USB_D_BUSY);

        // Liberamos el Mutex
        xSemaphoreGive(xUSB_Tx_Mutex);
    }
    return USB_D_OK;
}

void vTareaA(void *pvParameters){
    TickType_t PreviousWakeTime;
    char txBuffer[80]; // Buffer local para armar el mensaje
    UBaseType_t count = 1U;

    while(1) {
        PreviousWakeTime = xTaskGetTickCount();

        snprintf(txBuffer, sizeof(txBuffer),
            "--- TAREA B EJECUTÁNDOSE VEZ NUMERO %ld ---\r\n", count);
        count++;
        usb_transmit_buffer_safe(txBuffer);

        vTaskDelayUntil(&PreviousWakeTime, pdMS_TO_TICKS(100));
    }
}

void vTareaB(void *pvParameters){
    TickType_t PreviousWakeTime;
    char txBuffer[80]; // Buffer local para armar el mensaje
    UBaseType_t count = 1U;
```

```

while(1) {
    PreviousWakeTime = xTaskGetTickCount();

    snprintf(txBuffer, sizeof(txBuffer),
        "--- TAREA A EJECUTÁNDOSE VEZ NUMERO %ld ---\r\n", count);
    count++;
    usb_transmit_buffer_safe(txBuffer);

    vTaskDelayUntil(&PreviousWakeTime, pdMS_TO_TICKS(100));

}
}

```

Desafío 4

Funcionamiento: Reemplazar la sobrecarga de un Semáforo Binario global por una notificación directa a la tarea (Direct to Task Notification), optimizando la memoria y los ciclos de reloj.

Análisis:

```

void vTareaA(void *pvParameters){

    while(1) {
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
        HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_13);
        vTaskDelay(pdMS_TO_TICKS(200));

    }

}

static int8_t CDC_Receive_FS(uint8_t* Buf, uint32_t *Len)
{
    /* USER CODE BEGIN 6 */
    BaseType_t HigherPriorityTaskWoken = pdFALSE;
    vTaskNotifyGiveFromISR(xTarea_Terminal_Handle, &HigherPriorityTaskWoken);

    USBDCDC_SetRxBuffer(&hUsbDeviceFS, &Buf[0]);
    USBDCDC_ReceivePacket(&hUsbDeviceFS);

    portYIELD_FROM_ISR(HigherPriorityTaskWoken);
    return (USBD_OK);
    /* USER CODE END 6 */
}

```

Desafío 5

Funcionamiento: Utilizar el valor de notificación (Notification Value) para transmitir simultáneamente una señal de sincronización y un dato de estado de 32 bits desde una ISR hacia una tarea.

Análisis:

```
void vTareaComandos(void *pvParameters){

    uint32_t comand;

    while(1) {
        xTaskNotifyWait(0, ULONG_MAX, &comand, portMAX_DELAY);

        switch (comand) {
            case 0x01:
                usb_transmit_buffer_safe("[EVENTO] LED 1\r\n");
                HAL_GPIO_TogglePin(leds_param[0].GPIO_puerto, leds_param[0].GPIO_pin);
                HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_RESET);
                HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_RESET);
                HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_RESET);
                break;

            case 0x02:
                usb_transmit_buffer_safe("[EVENTO] LED 2\r\n");
                HAL_GPIO_TogglePin(leds_param[1].GPIO_puerto, leds_param[1].GPIO_pin);
                HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12, GPIO_PIN_RESET);
                HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_RESET);
                HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_RESET);
                break;

            case 0x03:
                usb_transmit_buffer_safe("[EVENTO] LED 3\r\n");
                HAL_GPIO_TogglePin(leds_param[2].GPIO_puerto, leds_param[2].GPIO_pin);
                HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_RESET);
                HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12, GPIO_PIN_RESET);
                HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_RESET);
                break;

            case 0x04:
                usb_transmit_buffer_safe("[EVENTO] LED 4\r\n");
                HAL_GPIO_TogglePin(leds_param[3].GPIO_puerto, leds_param[3].GPIO_pin);
                HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_RESET);
                HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_RESET);
                HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12, GPIO_PIN_RESET);
                break;

            default:
                usb_transmit_buffer_safe("[WARNING] Comando Invalido\r\n");
                break;
        }
    }
}

static int8_t CDC_Receive_FS(uint8_t* Buf, uint32_t *Len)
{
    /* USER CODE BEGIN 6 */
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    uint32_t value=0u;
}
```

```

if(xTarea_Comandos_Handle != NULL) {
    for(uint32_t i = 0; i < *Len; i++) {

        if(Buf[i] >= '1' && Buf[i] <= '4') {

            value = (uint32_t)(Buf[i] - '0'); // Le restamos el 0 ASCII
            xTaskNotifyFromISR(xTarea_Comandos_Handle, value, eSetValueWithOverwrite, &xHigherPriorityTaskWoken);
        }
    }
}

USBD_CDC_SetRxBuffer(&hUsbDeviceFS, &Buf[0]);
USBD_CDC_ReceivePacket(&hUsbDeviceFS);

portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
return (USBD_OK);
/* USER CODE END 6 */
}

```